

Lab Project

In this project, you will analyze and simulate **Sampson's Monks dataset**, a classic dataset in social network analysis.

By completing this project, you will:

- Work with matrices and lists in applied settings
- Practice row/column operations for network statistics
- Use functions, control flow, and libraries
- Understand the contrast between random vs. real-world networks

You will submit:

- A **.R file** with your code (well commented)
- A **README.md** describing your process, results, and reflections

Both files should be pushed to our GitHub Classroom by the due date (October 1 at 11:59PM).

Part 0 - Setting Up

Open your R app and install and load the necessary packages for this project, “network”, “ergm”, “igraph”, and “networkD3”. “ergm” contains the monks data.

Part 1 - Visualizing the Network

In this part, we will practice using **libraries**, converting between **matrix representations**, and thinking about directed vs. undirected graphs.

Using static and interactive plots, we are going to take a look at the monks data. First, let's load the data:

```
# Show all data sets in package
data(package = "ergm")

# Access the sampson data
data("samplike", package = "ergm")
samplike

# Create the vector and matrix
monk_mat <- as.matrix.network(samplike, matrix.type = "adjacency")
monk_names <- network.vertex.names(samplike)
```

- Create two network visualizations:

1. A static plot using `igraph::plot.igraph()`
2. An interactive plot using `networkD3::forceNetwork()`

- Before plotting, check what data structure your plotting function accepts:

- ▶ `igraph` accepts an adjacency matrix (convert from `monk_mat`). Use `as.matrix.network()` to get the adjacency matrix and convert it into `igraph`
- ▶ `networkD3` needs dataframes of nodes and edges, not a network format. Write code to transform the matrix into that format.

Part 2 - Summary Statistics on a Sociomatrix

Now we will apply our knowledge to practice **row/column operations**, **matrix means and sums**, and organizing outputs in **lists**.

- Choose one sociomatrix (samplk1 = “liking at time 1”, samplk2 = “liking at time 2”, samplk3 = “liking at time 3”) from the dataset other than the example used in lab.
- Compute summary measures:
 - Row sums (`rowSums()`) = out-degree
 - Column sums (`colSums()`) = in-degree
 - Mean tie strength (`rowMeans()`, `colMeans()`)
- Store results in a **list** that keeps track of different measures.
- Which monk is most liked? Which monk is least liked?
- Make a barplot of the out-degree statistic.

Part 3 - Generating a random sociomatrix

Next we will practice writing **functions**, using **for loops**, and **if/else control flow**.

Write a function that generates a random sociomatrix and print the randomly generated sociomatrix.

Part 4 - Analysis

In 3-5 sentences, compare **real monk network** (`monk_mat`) with your **randomly generated ones**:

- What patterns appear in the real monks network that are missing from the random ones?
- Why might real social networks show clustering, triangles, or central figures that random ones lack?

Submission

On GitHub Classroom, submit:

- `project.R` — R code with comments explaining each step
 - include Part 1, 2, 3
- `README.md` — with your answers, screenshots of plots, and reflections
 - include the short reflection from Part 4